SOFTWARE REQUIREMENTS SPECIFICATION

for

MOD-U-GO
(Moderating You on the Go)

An Online Examination and Proctoring Application

Version 3.0

Team Member	Roll Number
Ridham Shah	B24CS1064
Atharv Rahul Dhuvad	B24CS1085
Shrey Shah	B24CS1111
Pranav H Nair	B24CS1056

Submitted to: Dr. Romi Banerjee
Department of Computer Science & Engineering
IIT Jodhpur

February 27, 2026

Contents

1. Introduction **4**
1.1. Purpose . . . 4
1.2. Conventions and Standards . . . 4
1.2.1. Document Conventions . . . 4
1.2.2. Design and Coding Standards . . . 4
1.3. Intended Audience and Reading Suggestions . . . 5
1.4. Product Scope . . . 5
1.5. References . . . 6
1.6. Alpha Card Managers (Essence Kernels) . . . 6

2. Overall Description **7**
2.1. Product Perspective . . . 7
2.2. Product Functions . . . 7
2.3. User Classes and Characteristics . . . 8
2.4. Operating Environment . . . 8
2.5. Design and Implementation Constraints . . . 8
2.6. User Documentation . . . 9
2.7. Assumptions and Dependencies . . . 9

3. External Interface Requirements **10**
3.1. User Interfaces . . . 10
3.2. Hardware Interfaces . . . 10
3.3. Software Interfaces . . . 10
3.4. Communications Interfaces . . . 10

4. System Features **12**
4.1. Anti-Cheating and Proctoring . . . 12
4.2. Live Monitoring Dashboard . . . 14
4.3. Exam Creation and Management . . . 15
4.4. SLM-Based Grading . . . 15
4.5. Result and Video Retention Policy . . . 16

5. System Design **18**
5.1. Class Diagram . . . 18
5.2. Use Case Diagram . . . 18
5.3. Activity Diagram . . . 19
5.4. Sequence Diagram . . . 21
5.5. Component Diagram . . . 21
5.6. Composite Structure Diagram . . . 24
5.7. Deployment Diagram . . . 24
5.8. Timing Diagram . . . 25

6. Other Non-Functional Requirements **27**
6.1. Performance Requirements . . . 27
6.2. Safety Requirements . . . 27

6.3. Security Requirements 27
6.4. Software Quality Attributes 28

7. Project Management and Planning **29**
7.1. Traceability Matrix 29
7.2. Project Management Tool 31
7.2.1. Detailed Sprint Plan (8 Weeks – Spring Semester 2025–26) 31

8. Legal, Copyright, and Compliance Considerations **34**
8.1. Copyright 34
8.2. Data Privacy and Protection Compliance 34
8.3. Educational and Examination Compliance 34
8.4. AI and Algorithmic Compliance 34

A. Glossary **35**

1. Introduction

1.1. Purpose

This document specifies the software requirements for **MOD-U-GO** (Moderating You on the Go), a comprehensive online examination and automated proctoring platform. The system is designed to maintain academic integrity during remote assessments through an Edge-AI architecture that minimises bandwidth consumption, protects student privacy, and automates grading using Small Language Models (SLMs).

This SRS serves as the authoritative reference for developers, testers, project supervisors, and stakeholders throughout the development lifecycle. It defines functional requirements, non-functional constraints, interface specifications, and project management artefacts.

1.2. Conventions and Standards

1.2.1. Document Conventions

- **Bold text** is used for specific term definitions, table headers, and emphasis.
- `Monospace font` is used for code snippets, class names, function names, database fields, and API endpoints.
- Requirement identifiers follow the pattern **REQ-X** (e.g., REQ-2), where X is a unique numeric identifier.
- Test cases are labelled as **TC-X**.
- Priorities are classified as: **High** (essential for v1.0), **Medium** (targeted for v1.1), or **Low** (future scope).

1.2.2. Design and Coding Standards

Backend (Python / Node.js)

- Follow PEP-8 for Python code.
- Function naming: `snake_case` (e.g., `capture_frame`).
- Class naming: `PascalCase` (e.g., `StreamManager`).
- Variable naming: `camelCase`.
- Constants: `UPPER_SNAKE_CASE`.

Frontend (React)

- Component naming: `PascalCase`.

- Variable naming: `camelCase`.

Project Management

Trello is used for Kanban-style task tracking with columns: Backlog → In Progress → Review → Done.

Commenting Standards

- *File headers*: Every file must begin with a header indicating author, date, and purpose.
- *Function documentation*: Python uses Docstrings (`""" ... """`); Node.js / React uses JSDoc (`/** ... */`).
- *Inline comments*: Use `//` or `#` to explain complex logic; avoid restating the obvious.

Classification of Requirements

- **Functional Requirements**: Mandatory deliverables.
- **Non-Functional Requirements**: Elective enhancements and quality attributes.

1.3. Intended Audience and Reading Suggestions

- **Developers**: To understand functional requirements for implementing webcam access, SLM integration, and real-time WebSocket communication.
- **Project Supervisors**: To verify that the project meets academic and functional scope.
- **Testers**: To define test cases for anti-cheating mechanisms, grading accuracy, and system performance.

Reading Order: Section 2 provides the high-level product overview; Section 3 covers interface specifications; Section 4 details individual system features; Section 5 presents the UML system design models; Section 6 addresses non-functional requirements.

1.4. Product Scope

MOD-U-GO solves the challenge of maintaining integrity in online examinations. The platform provides three primary capabilities:

1. **Secure examination delivery**: Students take exams in a controlled browser environment with automated cheating detection. Keyboard shortcuts such as Ctrl+C, Ctrl+V, Ctrl+A, Ctrl+P, and PrtScn (screenshot) are blocked. Right-click context menus and browser developer tools are disabled. The exam runs in enforced full-screen mode.
2. **Automated proctoring**: A client-side Edge-AI model continuously monitors the student via webcam. A 5-second rolling buffer on the student's device is continuously

overwritten. Only when a violation is detected is the current 5-second clip uploaded to the server; all compliant footage is discarded locally and never transmitted.

3. **AI-driven assessment**: A Small Language Model (SLM) grades subjective and objective answers by semantic comparison with teacher-provided model answers.

1.5. References

- Essence Cards: Google Drive Link
- Slide Deck: Google Drive Link
- GitHub Repository: `https://github.com/pranav-codee/mod-u-go`
- Trello Board: `https://trello.com/b/bMAeKfUb/modugo`

1.6. Alpha Card Managers (Essence Kernels)

Team Member	Alpha Card(s)	Responsibility
Pranav H Nair	Stakeholders, Opportunity	Ensure stakeholder needs are met; verify the solution addresses the core business opportunity.
Ridham Shah	Requirements, Software System	Track requirement coherence; oversee technical architecture and system functionality.
Atharv Dhuvad	Work, Way of Working	Monitor task completion (Kanban flow); ensure the team follows agreed design/coding standards.
Shrey Shah	Team	Monitor team collaboration, skills, and availability to ensure smooth execution.

2. Overall Description

2.1. Product Perspective

MOD-U-GO is a standalone web application with a hybrid architecture comprising two decoupled layers:

1. **Edge Layer (Proctor Client)**: A lightweight browser-based React application that executes TensorFlow.js models locally on the student's device. This layer eliminates continuous video streaming; bandwidth is consumed only when a violation is detected.
2. **Service Layer (Backend)**: A microservices-based backend (FastAPI for the exam service; Node.js for real-time WebSocket proctoring) that handles authentication, exam lifecycle management, result storage, and SLM grading.

The system integrates with webcams (via the `MediaDevices` API), an SLM engine (locally hosted Llama/Mistral via Python), and a document database (MongoDB).

2.2. Product Functions

The major functions of the system are:

- **User Authentication**: Secure login with hashed passwords, JWT tokens, and OAuth2 support. Failed authentication attempts are rate-limited (see REQ-19).
- **Student Registration**: Controlled registration flow with retry limits (see REQ-18).
- **Exam Creation**: Teachers input questions, model answers, marks allocation, and per-question constraints.
- **Automated Proctoring**: Continuous local webcam inference using MediaPipe BlazeFace, WebGazer.js gaze tracking, tab-switch detection, and input restriction. Keyboard shortcuts (Ctrl+C/V/A/P, PrtScn) and screenshot tools are disabled.
- **Rolling Buffer & Anomaly Clip Delivery**: A 5-second local rolling buffer continuously retains and overwrites recent footage on the student's device. On violation detection, only the current 5-second clip is uploaded to the server and added to the teacher's review queue. Uploaded clips remain on the server until explicitly deleted by the teacher or until exam results are archived/deleted (see REQ-20, REQ-21).
- **Proctoring Window**: Proctoring begins a configurable interval before exam start and ends a configurable interval after submission (see REQ-16).
- **Live Monitoring Dashboard**: Event-based alerts with timestamps; teachers are not required to watch continuous feeds.
- **Automated Grading**: The SLM grades answers using a semantic similarity pipeline (see Section 4.4).

- **Result Management**: Teachers review, optionally override, and release scores. Violation clips are retained on the server until the teacher deletes them or until results are archived/deleted. Results follow a configurable retention and archival lifecycle (see REQ-20, REQ-21).

2.3. User Classes and Characteristics

- **Student**: Sufficient technical proficiency to use a web application and grant webcam permissions. Primary goal is to complete the exam without technical interruption.
- **Teacher**: Faculty member who creates exams, monitors integrity via the alert dashboard, and reviews/releases scores.
- **Administrator**: Overall authority responsible for managing all users, overseeing exam sessions, controlling access, and maintaining system integrity.

2.4. Operating Environment

- **Client Side**: Modern web browser (Chrome / Firefox / Edge) with WebRTC and WebGL support, functional webcam and microphone.
- **Server Side**: Python (FastAPI/Django) backend for SLM and REST endpoints; Node.js for WebSocket connections.
- **Database**: MongoDB (document store for schema flexibility).
- **Hardware**: Desktop or laptop with camera; minimum 4 GB RAM recommended for client-side AI inference.

2.5. Design and Implementation Constraints

1. **Bandwidth and Edge Computing Model**: The AI proctoring model runs locally using TensorFlow.js (WebGL backend). During normal exam conduct, zero video is uploaded. On violation detection, the buffered 5-second clip is uploaded. A burst upload speed of at least 1 Mbps is required.
2. **Client-Side Resource Constraints**: The application must operate within the browser memory limit (<4 GB for 64-bit browsers). The rolling buffer uses the `MediaRecorder` API with `video/webm;codecs=vp9` MIME type and `Blob` objects in a circular queue, maintaining a stable footprint of approximately 150 MB.
3. **Storage and Privacy**: The client-side rolling buffer continuously overwrites older segments every 5 seconds. If no violation is detected, the segment is discarded locally and never leaves the student's device. Only upon violation detection is the current 5-second clip uploaded to the server. Server-stored clips persist until the teacher explicitly deletes them or until the corresponding exam results are archived/deleted (see REQ-20, REQ-21). Assuming a 5% violation rate, 50 students, and an average of 5 violations per student: $2\ MB \times 5 \times 50 = 500\ MB$ total.
4. **Input Restrictions**: The exam interface disables keyboard shortcuts (Ctrl+C, Ctrl+V, Ctrl+A, Ctrl+P, PrtScn), right-click context menus, and browser developer tools via JavaScript event listeners. Special characters within answer fields **are**

permitted (see REQ-15).

5. **Authentication Limits**: Registration retries and authentication failures are bounded (see REQ-18, REQ-19).
6. **Camera Calibration**: Initial calibration is handled by the `CameraCalibrator` class within the `Web Client (AI Module)` component (see REQ-17).

2.6. User Documentation

- Online "How-to" guide for students (system compatibility check, webcam permission grant).
- Teacher guide for creating exams and interpreting violation alerts and cheat-probability scores.
- Administrator guide for user management and system settings.

2.7. Assumptions and Dependencies

- Users have a stable internet connection ($\geq$1 Mbps upload).
- Users possess working webcams and grant browser permissions.
- The selected SLM is capable of semantic comparison, not just keyword matching.
- All user data is stored securely with access restricted to authorised system components.
- Target browsers support the `MediaDevices`, `Page Visibility`, and `Fullscreen` APIs.

3. External Interface Requirements

3.1. User Interfaces

- **Login Screen**: Role selection (Teacher / Student / Admin) with rate-limited authentication.
- **Student Exam View**: Enforced full-screen mode, question display, countdown timer, webcam feed preview, and auto-save indicator.
- **Teacher Dashboard**:
  - *Exam Creation*: Form for questions, model answers, marks, word limits, and difficulty.
  - *Proctoring Dashboard*: Event-based alert list with student ID, violation type, timestamp, and link to flagged clip.
  - *Results View*: Table of student scores with "Override" and "Release" actions; configurable result availability period.
- **Admin Dashboard**: User management (edit, change role, ban); live proctoring overview; global results view.

3.2. Hardware Interfaces

The web client accesses the camera and microphone via `navigator.mediaDevices.getUserMedia` (HTML5 MediaDevices API). Video frames are processed locally using TensorFlow.js or MediaPipe within the browser. Camera calibration is performed by the `CameraCalibrator` class on session start (REQ-17).

3.3. Software Interfaces

- **Database**: MongoDB for user profiles, exam data, violation logs, and result records.
- **SLM Engine**: A locally hosted, fine-tuned model (Llama-2-7b quantised or Mistral) interfaced via Python; invoked asynchronously through a Celery/Redis worker queue.
- **Firebase Auth** (optional): OAuth2 / JWT token provider for production-grade authentication.

3.4. Communications Interfaces

- **HTTP/HTTPS**: Standard REST API for exam CRUD, user management, and result retrieval.

- **WebSockets**: Real-time push of violation alerts and exam-lock events from server to teacher dashboard.

4. System Features

4.1. Anti-Cheating and Proctoring

Priority: High. Monitors student behaviour in real time during the exam.

Stimulus/Response Sequences

- *Tab switch / focus loss*: System logs the event, issues an on-screen warning, records the current 5-second buffer segment, and queues the clip for teacher review.
- *No face or multiple faces detected*: System flags the interval as suspicious, uploads the clip, and sends an alert to the dashboard.
- *Gaze deviation > 5 continuous seconds*: System flags gaze-away event and uploads the clip.
- *Clipboard shortcut (Ctrl+C, Ctrl+V, Ctrl+A) or screenshot (PrtScn) attempt*: Event is blocked and logged.
- *Repeated violations exceeding configurable threshold N*: System locks the exam; teacher or administrator must unlock before the student can resume.
- *System pop-up / notification*: System logs and notifies the teacher dashboard; video is not recorded for this event alone.

Functional Requirements

REQ-1 (Focus Loss Detection)
The system shall detect when the exam window loses foreground focus using the HTML5 Page Visibility API (`document.hidden`) and `window blur/focus` event listeners.
Component: Web Client (Proctor Module) → *Class*: `EventMonitor` → *Function*: `handleVisibilityChange()`
Assigned to: Atharv Dhuvad

REQ-2 (Video Rolling Buffer)
The system shall continuously capture webcam input in contiguous **5-second segments** using the `MediaRecorder` API with `video/webm;codecs=vp9`. Each segment is retained as a `Blob` in a circular queue on the client device. Segments are automatically overwritten by the next segment if no anomaly is detected — no compliant footage is ever transmitted to the server. On anomaly detection, the current segment is uploaded to the server for teacher review.
Component: Web Client (Proctor Module) → *Class*: `StreamManager` → *Function*: `captureSegment()`
Assigned to: Atharv Dhuvad

REQ-3 (Exam Lock)

The system shall lock the exam after a configurable number of warnings (default: 3 tab switches). The exam can be unlocked only by a teacher or administrator.
Component: Exam Service → *Class*: `ExamSession` → *Function*: `lockExam()`
Assigned to: Ridham Shah

REQ-12 (Gaze Tracking)

The system shall use WebGazer.js to detect instances where the student's gaze deviates outside the screen bounding box for more than 5 continuous seconds, flagging the event as a potential violation. A "sustainment threshold" model is used to reduce false positives.
Component: Web Client (AI Module) → *Class*: `GazeTracker` → *Function*: `trackEyeMovement()`
Assigned to: Shrey Shah

REQ-13 (Pop-up Detection)

The system shall detect system-level pop-ups or notifications that cause the exam window to lose focus, logging them as potential distractions. No video is uploaded for this event type alone.
Component: Web Client → *Class*: `EventMonitor` → *Function*: `onWindowBlur()`
Assigned to: Atharv Dhuvad

REQ-14 (Input Restriction)

The system shall disable Ctrl+C, Ctrl+V, Ctrl+A, Ctrl+P, PrtScn, right-click context menus, and browser developer tools (`F12`) during the active exam window via JavaScript event listeners.
Component: Web Client (Proctor Module) → *Class*: `EventMonitor` → *Function*: `blockRestrictedInputs()`
Assigned to: Atharv Dhuvad

REQ-15 (Special Character Support)

The system shall permit entry of all standard Unicode special characters (e.g., mathematical symbols, punctuation, accented letters) in answer text fields. No character filtering beyond structural sanitisation (XSS prevention) is applied.
Component: Exam Service → *Class*: `AnswerField` → *Function*: `sanitiseInput()`
Assigned to: Ridham Shah

REQ-16 (Proctoring Window)

The system shall begin proctoring a configurable number of minutes before the scheduled exam start time (minimum: 0 min, maximum: 15 min; default: 5 min) and shall continue proctoring for a configurable number of minutes after the student submits (minimum: 0 min, maximum: 10 min; default: 2 min). These thresholds are configurable per exam by the teacher.
Component: Proctoring Service → *Class*: `ProctoringSession` → *Function*: `initProctoringWindow()`
Assigned to: Shrey Shah

REQ-17 (Camera Calibration)

The system shall perform camera calibration at the start of every proctoring session to normalise face bounding-box thresholds for the student's physical environment (lighting, distance). Calibration shall complete within 30 seconds.
Component: Web Client (AI Module) → *Class*: `CameraCalibrator` → *Function*: `calibrateCamera()`
Assigned to: Shrey Shah

REQ-18 (Registration Retries)

The system shall permit a maximum of **3 registration attempts** per email address per 24-hour window. Exceeding this limit shall result in a temporary lock of the email for registration, with a user-visible message and a countdown to retry.
Component: Auth Service → *Class*: `RegistrationService` → *Function*: `registerUser()`
Assigned to: Pranav H Nair

REQ-19 (Authentication Failures)

The system shall permit a maximum of **5 consecutive failed login attempts** per account. After 5 failures, the account shall be temporarily locked for 15 minutes. Persistent failures (>10 in 1 hour) shall trigger an administrator alert.
Component: Auth Service → *Class*: `AuthService` → *Function*: `handleLoginFailure()`
Assigned to: Pranav H Nair

4.2. Live Monitoring Dashboard

Priority: High. Allows teachers to review flagged incidents without requiring continuous live surveillance.

Stimulus/Response Sequences

- *Teacher opens Proctoring Dashboard*: System displays a list of active/completed exams with per-student violation summaries and warning counts.
- *Suspicious event detected*: System generates an alert entry containing student ID, timestamp, violation type, and link to the flagged video clip.
- *Pop-up occurs on student screen*: System logs the event and sends a non-intrusive dashboard notification; no video is recorded.

Functional Requirements

REQ-4 (Centralised Dashboard)

The system shall provide a dashboard listing students, warning counts, and violation summaries for each ongoing or completed exam.
Component: Web Client (Teacher Dashboard) → *Class*: `Teacher` → *Function*: `liveProctoring()`
Assigned to: Ridham Shah

REQ-5 (Event Alerts)

The system shall display event-based alerts (tab switch, face detection anomaly, pop-up) with timestamps and violation type, without displaying continuous live video feeds.
Component: Proctoring Service → *Class*: `ProctoringEvent` → *Function*: `detectSuspiciousEvent()`
Assigned to: Ridham Shah

4.3. Exam Creation and Management

Priority: Medium.

Stimulus/Response Sequences

- *Teacher submits exam form*: System validates all fields and saves questions, model answers, and constraints to the database.

Functional Requirements

REQ-6 (Question Types)
The system shall allow teachers to create the following question types: Multiple Choice Questions (MCQ), text-based short/long answers, and file uploads (handwritten diagrams or solutions).
Component: Exam Service → *Class*: `Teacher` → *Function*: `createExam()`
Assigned to: Ridham Shah

REQ-7 (Model Answer)
The system shall require a model answer for every text-based and MCQ question. The model answer is used by the SLM grading engine.
Component: Exam Service → *Class*: `Question` → *Function*: `createAnswers()`
Assigned to: Ridham Shah

REQ-11 (Question Constraints)
The system shall allow teachers to define per-question constraints: word limit, marks allocation, and difficulty level.
Component: Exam Service → *Class*: `Exam` → *Function*: `createExam()`
Assigned to: Ridham Shah

4.4. SLM-Based Grading

Priority: High. Automates scoring for text-based answers using semantic similarity.

Stimulus/Response Sequences

- *Student submits exam*: Each answer is enqueued in the Celery/Redis worker. The SLM asynchronously compares the student answer to the model answer and returns a normalised similarity score. The system converts this score to marks.

SLM Scoring Algorithm

The following protocol governs how the SLM assessment is converted to a numeric grade:

1. **Semantic Similarity Scoring**: The SLM (quantised Llama-2-7b or Mistral-7b) is prompted with the following structured template:

```
Given the model answer: ``{model_answer}'' and the student answer:
``{student_answer}'', rate the semantic similarity of the student
```

```
answer on a scale of 0.0 to 1.0, where 1.0 is a perfect match
in meaning.  Return only the numeric score.
```

2. **Score Normalisation**: The raw SLM similarity score $s \in [0.0, 1.0]$ is mapped to marks as: $marks = \lfloor s \times M \rceil$, where M is the maximum marks for the question and $\lfloor \cdot \rceil$ denotes rounding to the nearest integer.
3. **MCQ Grading**: For MCQ questions, exact match is performed programmatically without SLM invocation.
4. **Partial Credit**: For descriptive answers, partial credit is applied proportionally. No negative marking is applied by default (teacher-configurable).
5. **Edge Cases**:
   - Blank answers → score = 0.
   - SLM inference timeout (>30 s) → flagged for manual review; score = `PENDING`.
   - Off-topic or incoherent answers → SLM score typically < 0.1; teacher can override.
6. **Human Override**: Teachers may override any SLM-assigned score before result release (REQ-10).

Functional Requirements

REQ-8 (SLM Semantic Grading)
The SLM shall compare the semantic meaning of the student answer versus the model answer asynchronously via a Celery/Redis worker queue on the backend.
Component: AI Engine (Scoring Service) → *Class*: `AutoGrader` → *Function*: `generateRiskScore()`
Assigned to: Shrey Shah

REQ-9 (Score Conversion)
The system shall convert the SLM similarity score to a numerical mark using the formula in Section 4.4.
Component: Reporting Service → *Class*: `Report` → *Function*: `generateMarks()`
Assigned to: Shrey Shah

REQ-10 (Manual Override)
Teachers shall be able to manually override the SLM-assigned score before releasing results.
Component: Reporting (ReportGenerator) → *Class*: `Teacher` → *Function*: `generateReport()`
Assigned to: Ridham Shah

4.5. Result and Video Retention Policy

REQ-20 (Video Retention)
Violation video clips uploaded to the server shall be retained for the duration of the exam result lifecycle. Clips are automatically deleted when the corresponding exam results are archived or deleted (see REQ-21), unless a teacher or administrator explicitly deletes them earlier. Compliant (non-violation) footage is never transmitted to or stored on the server; it is continuously overwritten in the client-side rolling

buffer and discarded locally.

REQ-21 (Result Availability)

Exam results shall remain accessible to students for a configurable period (default: **6 months after release**). After this period, results are archived (read-only, accessible to administrators and teachers only). Archived results are retained for a minimum of **3 years** for academic record purposes.

5. System Design

This chapter presents the UML design models for the MOD-U-GO system architecture. Each diagram captures a distinct structural or behavioural view of the system, collectively providing a comprehensive blueprint for implementation.

5.1. Class Diagram

The class diagram (Figure 5.1) defines the static structure of the system, including all classes, attributes, methods, inheritance hierarchies, associations, and enumerations. Every functional requirement (REQ-1 through REQ-21) maps to at least one class or method in this diagram.

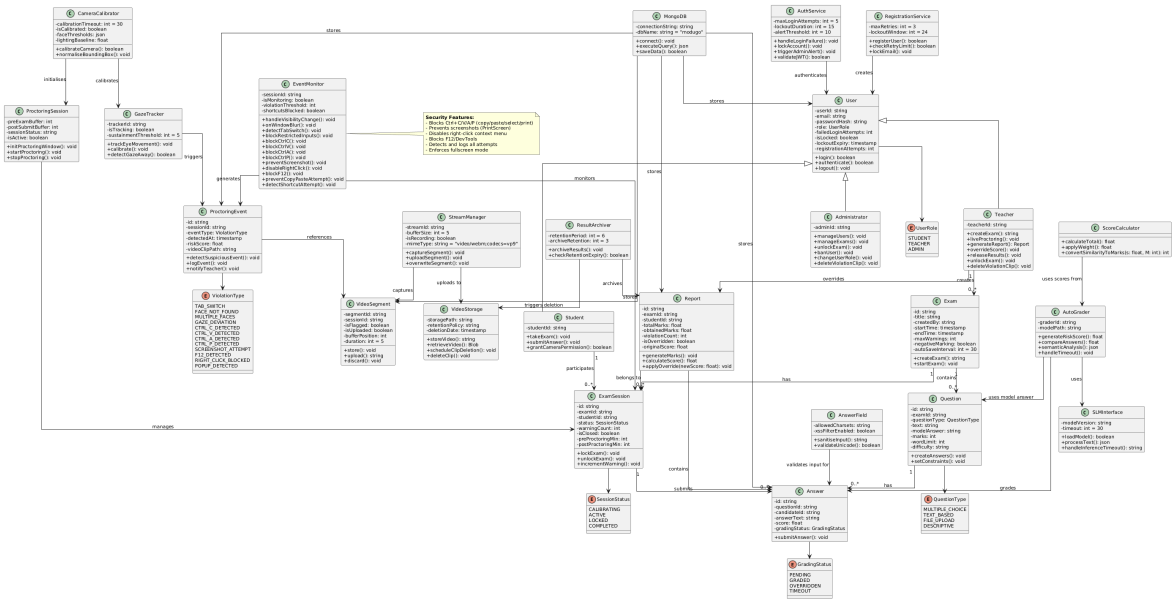

Figure 5.1.: Class Diagram — Static structure of the MOD-U-GO system showing all classes, relationships, and enumerations.

5.2. Use Case Diagram

The use case diagram (Figure 5.2) identifies the primary actors and their interactions with the system:

- **Student**: Logs in (with two-factor authentication), takes exams (which includes viewing questions, submitting answers, and auto-saving progress), views results, and accesses help resources.
- **Proctor (Teacher)**: Monitors students during exams, which includes detecting suspicious activity (REQ-1, REQ-12, REQ-13, REQ-14) and generating proctoring reports (REQ-5).

- **Administrator**: Manages exams, manages users (role changes, bans), and views reports.
- **System Actors**: The Auth Service handles authentication and token validation (REQ-18, REQ-19); the Database persists all exam, answer, and violation data.

Key relationships shown:

- *Login include Two-Factor Authentication* — every login requires 2FA.
- *Take Exam include View Questions*, *Submit Answers*, *Auto-Save Progress* — these are mandatory sub-flows of exam-taking.
- *Monitor Student include Detect Suspicious Activity*, *Generate Proctor Report* — monitoring always includes detection and reporting.
- *Take Exam extend Get Help* — students may optionally access help during the exam.

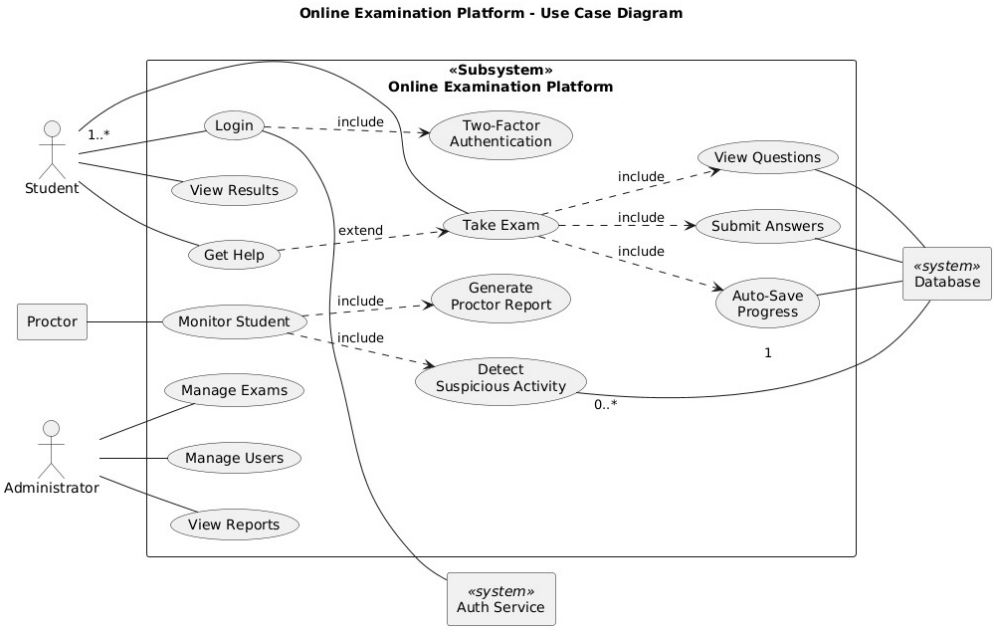

Figure 5.2.: Use Case Diagram — Actors, use cases, and include/extend relationships for the online examination platform.

5.3. Activity Diagram

The activity diagram (Figure 5.3) illustrates the three primary workflows of the system:

- **User Authentication**: Registration retry logic (REQ-18), login failure handling (REQ-19), and credential validation flow.
- **Exam Lifecycle**: Role-based routing (Student, Teacher, Admin), exam creation, exam session join, and dashboard navigation.
- **Proctoring and Monitoring**: Real-time monitoring loop, suspicious event detection (REQ-1, REQ-12, REQ-13), event flagging, violation clip upload (REQ-2), and proctoring report generation.

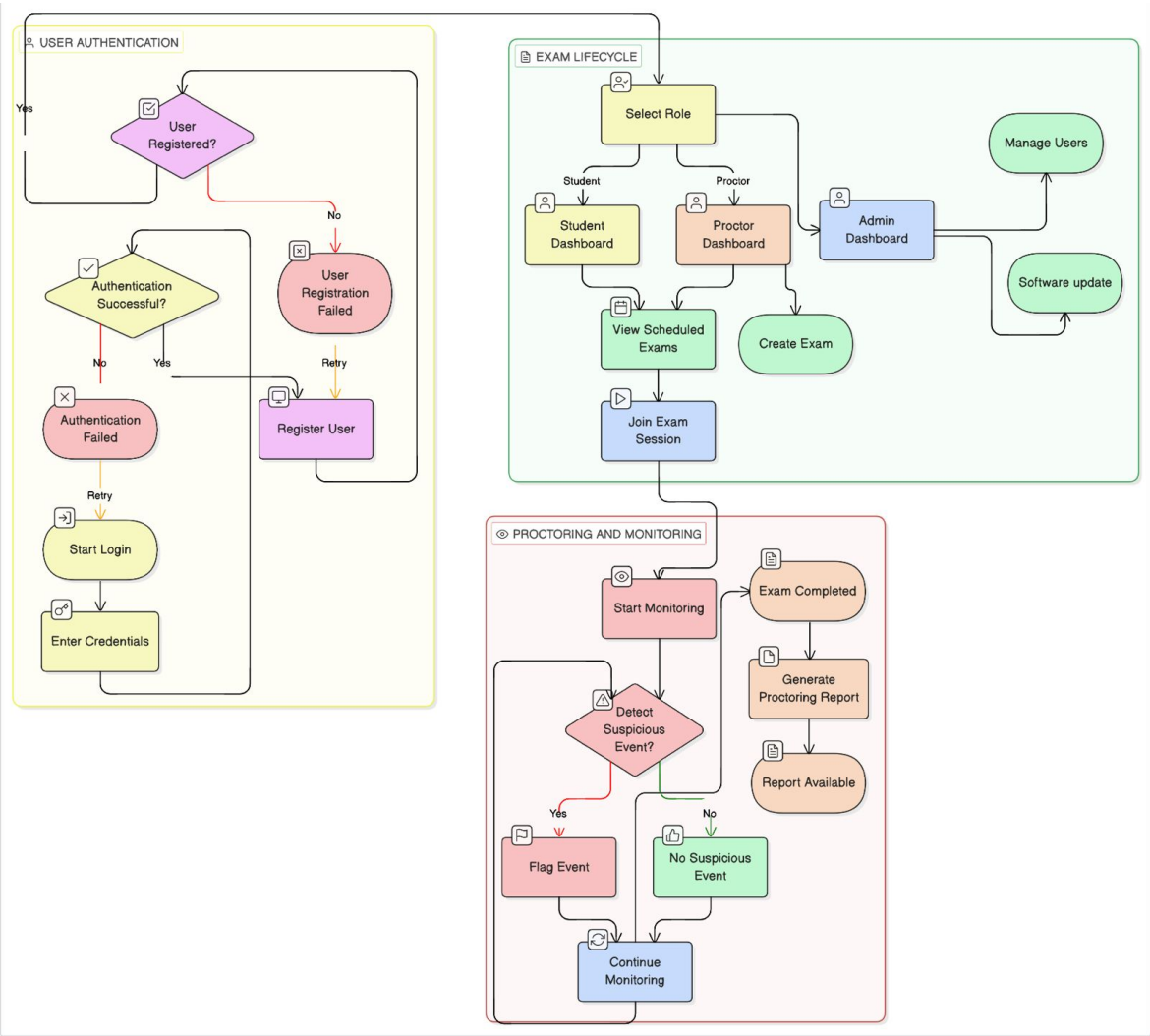

Figure 5.3.: Activity Diagram — User authentication, exam lifecycle, and proctoring workflows.

5.4. Sequence Diagram

The sequence diagram (Figure 5.4) traces the complete end-to-end interaction flow across all system components for a single student exam session. The diagram is organised into six sequential phases:

1. **Login Phase**: The student enters credentials in the Browser, which sends a login request through the Gateway to the Auth Service. The Auth Service queries the Database for user data, performs 2FA verification (REQ-19), generates a JWT token, and returns authentication confirmation. The Browser displays the student dashboard.
2. **Exam Loading Phase**: The student requests an exam. The Gateway validates the JWT token via the Auth Service, then forwards the request to the Exam Service. The Exam Service queries the Database for exam data, loads questions, processes the exam structure, and returns the exam content to the Browser for display.
3. **Proctoring Initialisation**: The Browser initialises proctoring by communicating with the Proctor Service, which triggers camera calibration (REQ-17). Once calibration completes within 30 seconds, proctoring becomes active and the configurable pre-exam window begins (REQ-16).
4. **Exam Taking Phase**: The student answers questions. A loop runs every 30 seconds (SR-1: Auto-Save) where the Browser saves progress through the Exam Service to the Database, with confirmation returned to the student.
5. **Proctoring Events**: During the exam, the Proctor Service continuously detects suspicious events (REQ-1, REQ-12, REQ-13, REQ-14). Detected events are logged to the Database, and real-time alert notifications are pushed to the Teacher dashboard via WebSocket (REQ-5).
6. **Submission Phase**: The student submits the exam. Answers flow through the Gateway to the Exam Service, which validates and saves final answers to the Database. Upon successful submission, proctoring stops and the Proctor Service generates a proctoring report. Results are returned to the Browser.

5.5. Component Diagram

The component diagram (Figure 5.5) shows the high-level modular architecture of the system, organised into the following service boundaries:

- **Web Client**: Contains the Student, Teacher, StreamManager, GazeTracker, and EventMonitor components that run on the client device (Edge Layer).
- **API Gateway**: Routes HTTP and WebSocket requests through `RequestRouter` and `AuthMiddleware` for role-based access control (SEC-2).
- **Exam Service**: Manages `Exam`, `Question`, `ExamSession`, and `Answer` entities.
- **AI Engine**: Hosts the `AutoGrader`, `SLMInterface`, and `ModelLoader` for semantic grading (REQ-8, REQ-9).
- **Proctoring Service**: Processes `ProctoringEvent` entities, runs `ViolationDetector`, and manages `AlertManager` for real-time teacher notifications (REQ-5).
- **Auth Service**: Handles `UserAuth`, `SessionManager`, and `TokenValidator` for au-

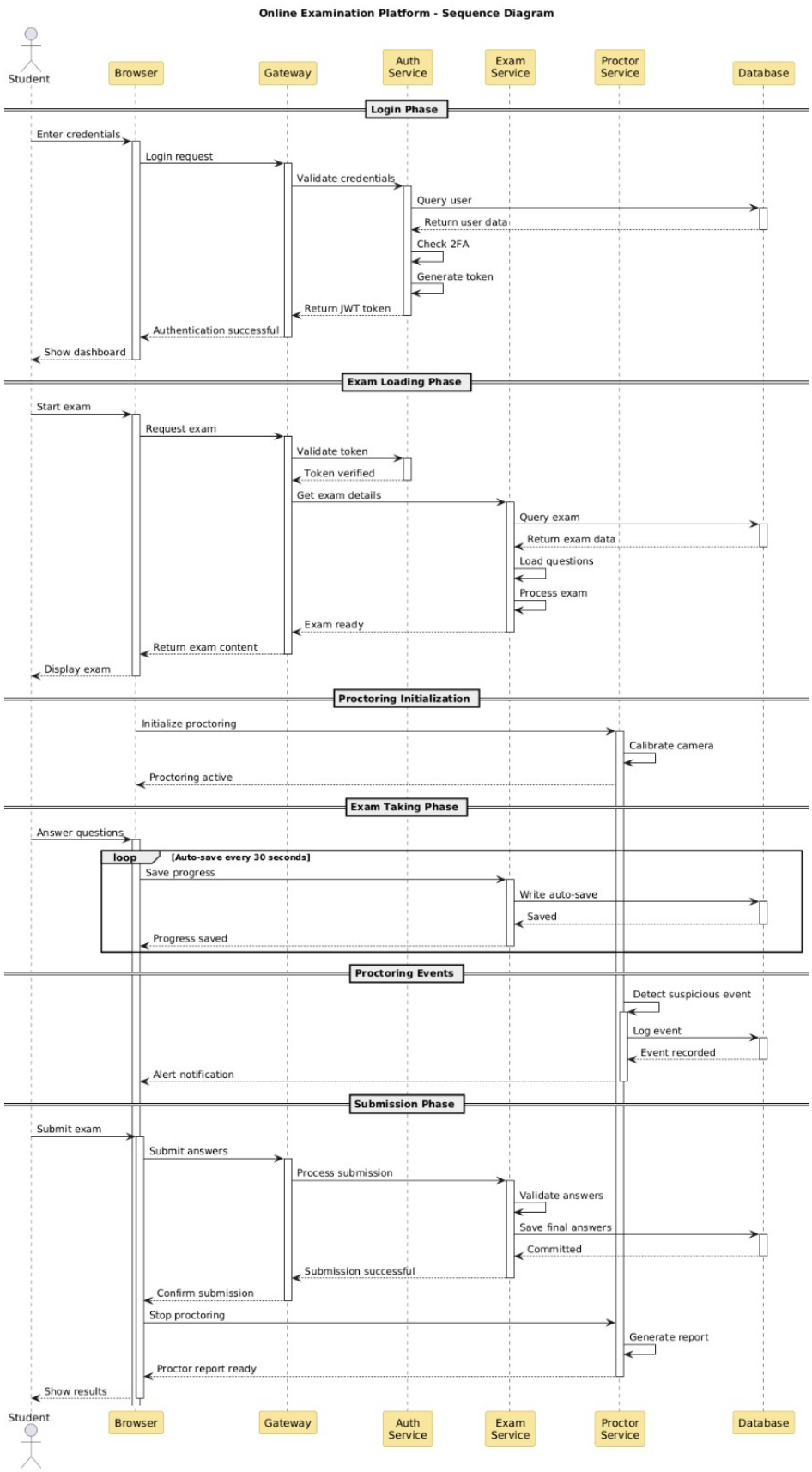

Figure 5.4.: Sequence Diagram — End-to-end interaction flow for a student exam session across Browser, Gateway, Auth Service, Exam Service, Proctor Service, and Database.

thentication and session management (REQ-18, REQ-19).

- **Reporting Service**: Generates `Report`, `ScoreCalculator`, and `ReportGenerator` outputs (REQ-9, REQ-10).
- **Storage Service**: Manages `VideoStorage` and `FileUploadHandler` for violation clip retention (REQ-20).
- **Database Service**: Central MongoDB instance for persistent data storage.

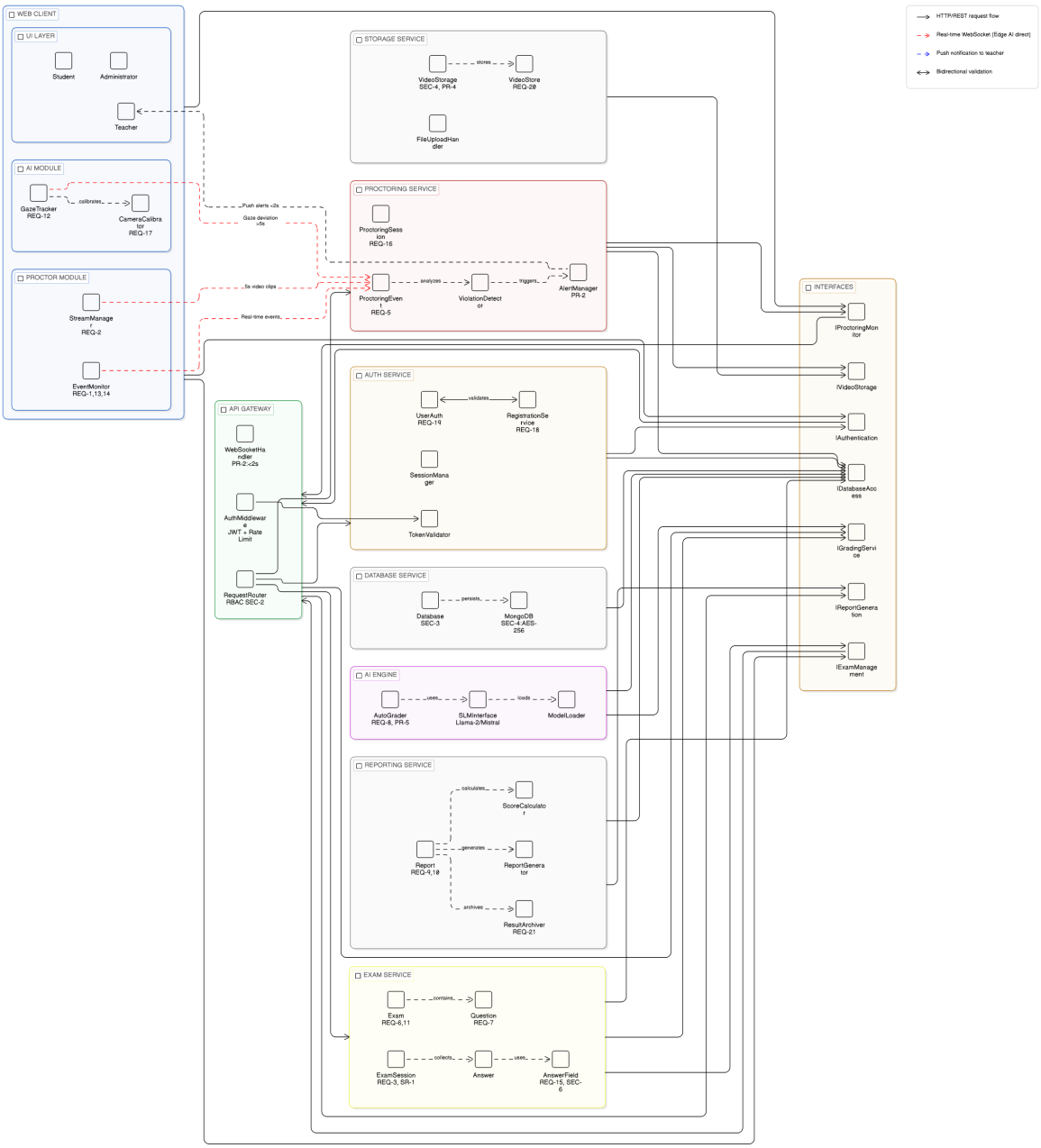

Figure 5.5.: Component Diagram — Microservices architecture showing service boundaries, interfaces, and inter-service communication.

5.6. Composite Structure Diagram

The composite structure diagram (Figure 5.6) provides a refined view of the internal composition of each service, showing how components are nested within their respective service packages and how data flows between them:

- HTTP/WebSocket requests flow from the Web Client through the API Gateway.
- The API Gateway routes authenticated requests to the Exam Service, Reporting Service, Proctoring Service, and Auth Service.
- Violation events trigger real-time WebSocket alerts from the Proctoring Service back to the Teacher dashboard.
- The AI Engine receives answers from the Exam Service for semantic grading and returns scores to the Reporting Service.
- All services persist data to the central Database.
- Violation clips are uploaded from the Proctoring Service to Video Storage.

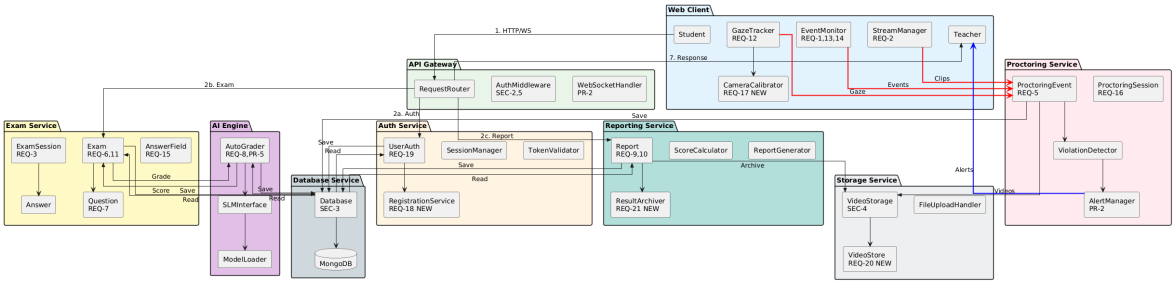

Figure 5.6.: Composite Structure Diagram — Internal composition of services with data flow paths between components.

5.7. Deployment Diagram

The deployment diagram (Figure 5.7) maps the system's components to physical and virtual infrastructure nodes:

- **Client Devices**: Student devices run Edge-AI components (EventMonitor, StreamManager, GazeTracker) locally in the browser. Teacher and Admin devices host their respective dashboards.
- **Load Balancer**: Nginx distributes incoming traffic across backend service instances.
- **API Gateway**: Hosts `RequestRouter`, `AuthMiddleware`, `WebSocketHandler`, and `GatewayService`.
- **Backend Services**: Exam Service, Auth Service, AI Engine, Proctoring Service, and Reporting Service are deployed as independent microservice containers.
- **Database Server**: MongoDB instance for persistent storage.
- **Storage Server**: Dedicated node for `VideoStorage` and `FileStorage` to handle violation clip retention (REQ-20).

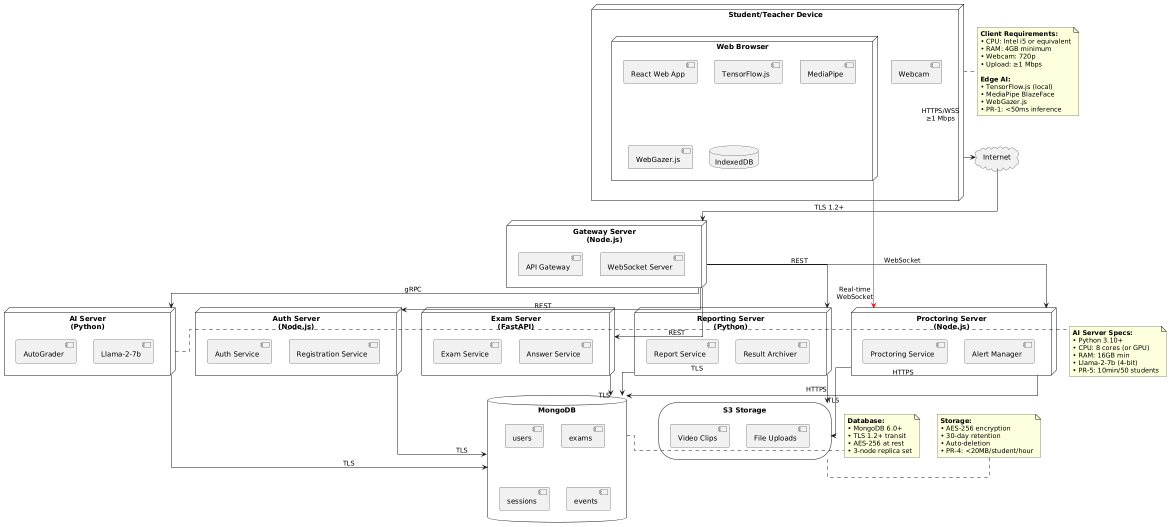

Figure 5.7.: Deployment Diagram — Physical and virtual infrastructure mapping of all system components.

5.8. Timing Diagram

The timing diagram (Figure 5.8) visualises the state transitions of all major system components over time during a complete exam session, with a time scale of 1 unit = 1000 ms. The diagram tracks six concurrent lifelines:

- **Browser**: Transitions through Idle → SendingLogin → WaitingAuth → Authenticated → RequestingExam → LoadingExam → TakingExam → AutoSaving → Submitting → ViewingResults. The auto-save state recurs every 30 seconds (SR-1).
- **Gateway**: Transitions through Idle → ReceivingRequest → Routing → ForwardingToken → ReceivingExamReq → ForwardingExam → ReceivingSubmit → Processing → SendingResponse. Shows the API Gateway's role as a stateless request router (SEC-2).
- **Auth**: Transitions through Idle → ValidatingCredentials → Checking2FA → GeneratingToken → TokenGenerated → ValidatingToken → Verified. Maps to REQ-18 and REQ-19 for authentication flow.
- **ExamSvc**: Transitions through Idle → ReceivingRequest → QueryingDB → LoadingQuestions → ProcessingExam → ExamReady → AutoSave → Monitoring → ReceivingAnswers → ValidatingAnswers → Saved. Maps to REQ-6, REQ-7, REQ-11.
- **Proctor**: Transitions through Inactive → Initialising → Calibrating → Monitoring → DetectingEvent → ProcessingAlert → Stopping → GeneratingReport. The Calibrating state corresponds to REQ-17 (30-second camera calibration). The Monitoring → DetectingEvent loop maps to REQ-1, REQ-12, REQ-13, REQ-14.
- **Database**: Transitions through Idle → QueryingUser → ReturningUser → QueryingExam → ReturningExam → WritingAutoSave → WritingEvent → WritingAnswers → Committed. Shows the database as a passive responder to service queries.

Key latency observations from the diagram:

- Authentication completes within ∼12 seconds (states 2–14).
- Exam loading completes within ∼8 seconds (states 14–22), satisfying PR-2.

- Proctoring initialisation and calibration span ∼6 seconds (states 20–26), well within the 30-second REQ-17 limit.
- The monitoring phase runs continuously from state 28 until submission.

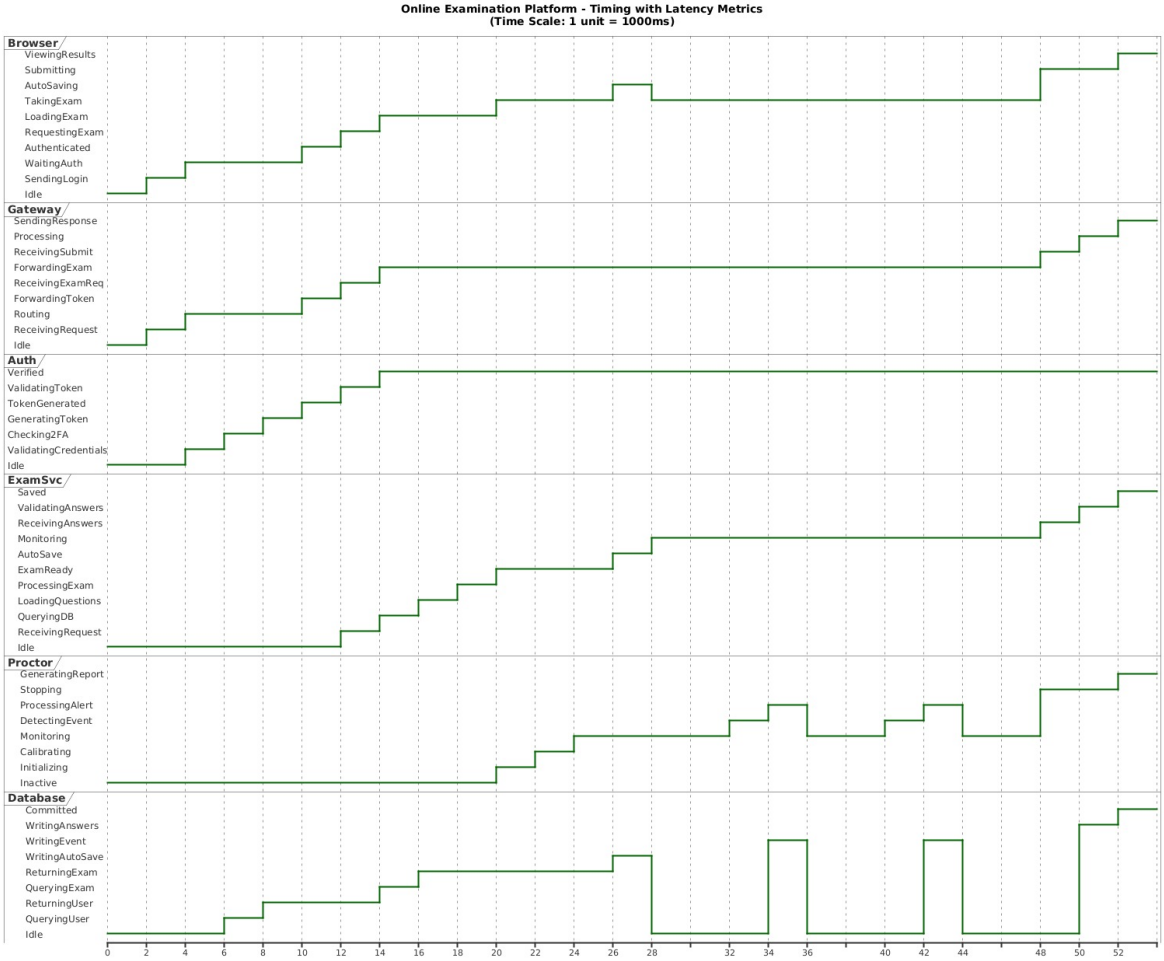

Figure 5.8.: Timing Diagram — State transitions of Browser, Gateway, Auth, Exam Service, Proctor, and Database over a complete exam session (1 unit = 1000 ms).

6. Other Non-Functional Requirements

6.1. Performance Requirements

- **PR-1 (Video Latency)**: Client-side AI inference latency shall not exceed 50 ms on standard consumer hardware (Intel i5 or equivalent) to ensure real-time feedback.
- **PR-2 (Dashboard Latency)**: Video alert delivery to the teacher dashboard shall not exceed 2 seconds from violation detection.
- **PR-3 (Concurrency)**: The system shall support at least 50 concurrent exam sessions without server-side CPU utilisation exceeding 70%.
- **PR-4 (Bandwidth)**: Total data consumption per student per 1-hour exam shall not exceed 20 MB under normal conditions (5% violation rate).
- **PR-5 (SLM Grading Throughput)**: The SLM grading queue shall process all pending answers within 10 minutes of exam submission for a class of 50 students.

6.2. Safety Requirements

- **SR-1 (Auto-Save)**: If the network connection drops, the system shall auto-save the student's progress locally (IndexedDB) every 30 seconds to prevent data loss, synchronising with the server upon reconnection.
- **SR-2 (Graceful Degradation)**: If the webcam becomes unavailable mid-exam, the system shall log the event and notify the teacher rather than immediately locking the exam.

6.3. Security Requirements

- **SEC-1 (Password Hashing)**: All passwords shall be hashed using bcrypt with a cost factor $\geq$12.
- **SEC-2 (Access Control)**: Students shall not be able to access "Create Exam" or "Grade" API routes. Role-based access control (RBAC) is enforced at the API gateway.
- **SEC-3 (Data Integrity)**: Exam answers shall be encrypted in transit using TLS 1.2 or higher.
- **SEC-4 (Photo / Video Encryption)**: All stored violation clips shall be encrypted at rest (AES-256) and accessible only to authorised users (teacher, admin) through controlled access mechanisms.
- **SEC-5 (Rate Limiting)**: API endpoints shall implement three-tier rate limiting: general API (100 req/min), authentication (5 req/min), and exam submission (1 req/exam session).

- **SEC-6 (Input Sanitisation)**: All text inputs shall be sanitised to prevent XSS and injection attacks while preserving allowed special characters (REQ-15).

6.4. Software Quality Attributes

- **Reliability**: The proctoring mechanism shall not generate excessive false positives (target: <5% false positive rate for face-absence detection; <10% for gaze-tracking).
- **Usability**: The interface shall be navigable by non-technical users within 5 minutes without training.
- **Maintainability**: The codebase shall follow a modular microservices architecture with clear separation of concerns; all interfaces shall be documented.
- **Privacy**: Biometric processing shall occur 100% locally on the student's device. No continuous video stream shall be transmitted to the server. Only 5-second violation clips are uploaded upon detection; all other footage is discarded locally. Server-stored clips are deleted when the teacher requests deletion or when exam results are archived/deleted.
- **Compliance**: For deployment in India, the system shall follow the Digital Personal Data Protection (DPDP) Act. For international deployment, GDPR principles apply.

7. Project Management and Planning

7.1. Traceability Matrix

The complete implementation traceability matrix, including detailed development tracking, status updates, team member assignments, and requirement change history, is maintained in the following Google Sheets document:

Traceability Matrix URL:
https://docs.google.com/spreadsheets/d/1FWPrnm5SeNddAwLhSmrTlVHAz3RQ6PGEznJakj6x6Ac/edit?gid=0#gid=0

This spreadsheet provides real-time tracking of:

- Requirement implementation status and progress
- Component and function-level development details
- Team member assignments and responsibilities
- Start and completion dates for each implementation task
- Requirement update history with change rationale
- Testing and integration status

For the most up-to-date implementation status, please refer to the online spreadsheet which is maintained throughout the development lifecycle. A summary of the requirements traceability is provided in Table 7.1.

Table 7.1.: Requirements Traceability Matrix

Requirement ID	Source	Component	Class / Function	Priority
Functional Requirements				
REQ-1, REQ-2, REQ-13, REQ-14	Section 4.1	Web Client (Proctor Module)	EventMonitor, StreamManager	High
REQ-3 to REQ-7, REQ-10, REQ-11, REQ-15, REQ-21	Section 4.1, Section 4.3, Section 4.4, Section 4.5	Exam Service, Reporting	ExamSession, Teacher, Question, AnswerField, ResultArchiver	High / Medium
REQ-4, REQ-5	Section 4.2	Web Client (Teacher Dashboard), Proctoring Service	Teacher, ProctoringEvent	High

Continued on next page...

Requirement ID	Source	Component	Class / Function	Priority
REQ-8, REQ-9	Section 4.4	AI Engine, Reporting	AutoGrader, Report	High
REQ-12, REQ-16, REQ-17	Section 4.1	Proctoring Service, Web Client (AI)	GazeTracker, ProctoringSession, CameraCalibrator	High
REQ-18, REQ-19	Section 4.1	Auth Service	Registration-Service, AuthService	High
REQ-20	Section 4.5	Storage Service	VideoStorage, ResultArchiver	High
Non-Functional Requirements — Performance				
PR-1	Section 6.1	Web Client (AI Module)	GazeTracker, CameraCalibrator	High
PR-2	Section 6.1	Proctoring Service, API Gateway	AlertManager, WebSocketHandler	High
PR-3	Section 6.1	All backend services	ExamSession, AuthService	High
PR-4	Section 6.1	Web Client (Proctor Module)	StreamManager	Medium
PR-5	Section 6.1	AI Engine	AutoGrader, SLMInterface	High
Non-Functional Requirements — Safety				
SR-1	Section 6.2	Web Client, Exam Service	ExamSession (IndexedDB auto-save)	High
SR-2	Section 6.2	Web Client (AI Module), Proctoring Service	CameraCalibrator, ProctoringEvent	Medium
Non-Functional Requirements — Security				
SEC-1	Section 6.3	Auth Service	UserAuth (bcrypt hashing)	High
SEC-2	Section 6.3	API Gateway	AuthMiddleware, RequestRouter	High
SEC-3	Section 6.3	All services (infrastructure)	TLS 1.2+ at transport layer	High
SEC-4	Section 6.3	Storage Service	VideoStorage (AES-256)	High
SEC-5	Section 6.3	API Gateway	RequestRouter (rate limiter)	Medium
SEC-6	Section 6.3	Exam Service	AnswerField	High

Requirement Update Log

The detailed requirement update log, including change dates, descriptions, and rationale for each modification, is maintained in the traceability spreadsheet linked above. A total of **15**

requirement updates have been recorded across versions 1.0–3.0, covering enhancements to the rolling buffer (REQ-2), dashboard design (REQ-4, REQ-5), input restrictions (REQ-14), and the addition of new requirements for security hardening (REQ-18, REQ-19), retention policies (REQ-20, REQ-21), camera calibration (REQ-17), proctoring windows (REQ-16), and special character support (REQ-15).

7.2. Project Management Tool

Trello (https://trello.com/b/bMAeKfUb/modugo) is used for Kanban-style task tracking. Board columns: Backlog → Development → Testing → Completed.

7.2.1. Detailed Sprint Plan (8 Weeks – Spring Semester 2025–26)

The sprint plan prioritises the data layer first (Foundation), then the core proctoring value (Security), and finally the intelligence layer (SLM Assessment). This schedule ensures every functional and non-functional requirement is met within the 8-week timeline.

Sprint	Dates	Objectives	Key Deliverables & Requirement Mapping
1	Jan 16 – Jan 29 (Weeks 1–2)	**Foundation**: Database schema, Authentication, and Exam Setup.	• **DB Schema**: MongoDB collections for users, exams, and proctoring logs (Ridham Shah). • **Auth Service**: Registration with 3-attempt limit **(REQ-18)**; login lockout after 5 failures **(REQ-19)** (Pranav H Nair). • **Password Hashing**: Bcrypt with cost factor ≥12 **(SEC-1)** (Pranav H Nair). • **Access Control**: Role-Based Access Control at API gateway **(SEC-2)** (Pranav H Nair). • **Rate Limiting**: Three-tier limits for General, Auth, and Submission endpoints **(SEC-5)** (Pranav H Nair). • **Exam Management**: CRUD for question types **(REQ-6)**, model answers **(REQ-7)**, and per-question constraints **(REQ-11)** (Ridham Shah).

Sprint	Dates	Objectives	Key Deliverables & Requirement Mapping
2	Jan 30 – Feb 12 (Weeks 3–4)	**Core Proctoring**: Webcam access, Rolling buffer, and Input restrictions.	• **Rolling Buffer**: WebRTC-based 5-second local circular buffer **(REQ-2)** (Atharv Dhuvad). • **Focus Detection**: Tab switch / focus loss detection via Page Visibility API **(REQ-1)** (Atharv Dhuvad). • **Pop-up Detection**: System notification / window blur logging **(REQ-13)** (Atharv Dhuvad). • **Input Restriction**: Block Ctrl+C/V/A/P, PrtScn, right-click, F12 **(REQ-14)** (Atharv Dhuvad). • **Calibration**: 30-second camera calibration routine **(REQ-17)** (Shrey Shah). • **Proctoring Window**: Configurable pre/post-exam proctoring interval **(REQ-16)** (Shrey Shah). • **TLS Enforcement**: TLS 1.2+ for all data in transit **(SEC-3)** (Pranav H Nair). • **Bandwidth**: Stream optimisation to stay <20 MB per hour **(PR-4)** (Atharv Dhuvad).
3	Feb 13 – Feb 26 (Weeks 5–6)	**Advanced Integrity**: Gaze tracking, Alerting, and Exam Lockdown.	• **Gaze Tracking**: WebGazer.js; flag deviation >5 continuous seconds **(REQ-12)** (Shrey Shah). • **Exam Lock**: Auto-lock after configurable violation threshold **(REQ-3)** (Ridham Shah). • **Dashboard**: Centralised teacher view with per-student warning counts **(REQ-4)** (Ridham Shah). • **Event Alerts**: Timestamped alerts for tab switch, face anomaly, pop-up **(REQ-5)** (Ridham Shah). • **AI Latency**: Client-side inference tuned to <50 ms **(PR-1)** (Shrey Shah). • **Alert Latency**: Real-time alert delivery to dashboard in <2 seconds **(PR-2)** (Ridham Shah). • **Graceful Degradation**: Log and notify teacher on webcam failure; no immediate lock **(SR-2)** (Ridham Shah).

Sprint	Dates	Objectives	Key Deliverables & Requirement Mapping
4	Feb 27 – Mar 12 (Weeks 7–8)	**Intelligence & Reporting**: SLM Grading, Archival, and Safety.	• **SLM Grading**: Async semantic similarity pipeline via Celery/Redis **(REQ-8)** (Shrey Shah). • **Score Conversion**: Similarity score to numeric marks formula **(REQ-9)** (Shrey Shah). • **Manual Override**: Teacher override of SLM scores before release **(REQ-10)** (Ridham Shah). • **Special Characters**: Unicode support in answer fields with XSS prevention **(REQ-15, SEC-6)** (Ridham Shah). • **Video Retention**: Clips retained until teacher deletion or result archival **(REQ-20)** (Pranav H Nair). • **Result Archival**: 6-month student access; 3-year admin/teacher archive **(REQ-21)** (Pranav H Nair). • **Video Encryption**: AES-256 at rest for stored violation clips **(SEC-4)** (Pranav H Nair). • **Concurrency**: Support 50+ simultaneous exam sessions **(PR-3)** (Atharv Dhuvad). • **Grading Throughput**: Process 50-student class within 10 minutes **(PR-5)** (Shrey Shah). • **Auto-Save**: 30-second local auto-save via IndexedDB on network drop **(SR-1)** (Atharv Dhuvad).

8. Legal, Copyright, and Compliance Considerations

8.1. Copyright

Original source code written for MOD-U-GO is automatically protected under copyright law. Open-source libraries (NumPy, Pandas, TensorFlow, React, etc.) are used under their respective licences with proper attribution. Project documentation is the original intellectual property of the authors.

8.2. Data Privacy and Protection Compliance

Personal and sensitive data (student details, exam responses, webcam clips) are collected only when necessary and require explicit user consent. Webcam footage is processed entirely on the student's device; only 5-second violation clips are uploaded to the server. These clips persist on the server until the teacher explicitly deletes them or until the corresponding exam results are archived or deleted (see REQ-20, REQ-21). No compliant footage is ever stored on the server. For deployment in India, the system follows the Digital Personal Data Protection (DPDP) Act. For international deployment, GDPR applies, including user rights to access and deletion.

8.3. Educational and Examination Compliance

Proctoring mechanisms are transparent and communicated to users before the exam starts. Automated proctoring decisions are advisory; human review by teachers is always possible (REQ-10). The system avoids disadvantaging users due to accessibility or hardware limitations (gaze calibration per session – REQ-17).

8.4. AI and Algorithmic Compliance

The SLM grading system is explainable (scoring rationale is logged) and interpretable by teachers. False positives in proctoring are minimised through calibration and sustainment thresholds (REQ-12, REQ-17).

A. Glossary

SLM Small Language Model — a lightweight AI model optimised for specific tasks (e.g., text semantic comparison) with lower resource requirements than large foundational models.

Proctoring The act of monitoring students during an exam to ensure academic integrity.

Rolling Buffer A fixed-size circular memory buffer that retains only the most recent N seconds of video, overwriting older data continuously.

Edge-AI AI inference executed on the client device (edge) rather than a remote server.

WebRTC Web Real-Time Communication — a browser API enabling peer-to-peer audio/video communication.

SRS Software Requirements Specification.

REST API Representational State Transfer — a stateless web service architecture.

RBAC Role-Based Access Control — access permissions granted based on user role.

JWT JSON Web Token — a compact, self-contained token for secure information exchange.

DPDP Digital Personal Data Protection Act (India, 2023).

GDPR General Data Protection Regulation (EU, 2018).

Bibliography

[1] MOD-U-GO GitHub Repository. Available: https://github.com/pranav-codee/mod-u-go.

[2] MOD-U-GO Trello Board. Available: https://trello.com/b/bMAeKfUb/modugo.

[3] System Design Presentation. Available: https://drive.google.com/drive/folders/1rocdYRpOxTREtwwUFji-blPgp8DVfNgO?usp=sharing.

[4] Project Essence Kernel Definitions. Available: https://drive.google.com/file/d/1zTIqEkrZT7t3wpivwKdJ9FWOK7-MEOw-/view?usp=sharing.

[5] "AI Online Exam Proctoring System," *International Journal of Innovative Research in Technology (IJIRT)*, vol. 8, no. 9, pp. 120–125. Available: https://ijirt.org/publishedpaper/IJIRT189267_PAPER.pdf.

[6] C. Lachner, T. Rausch, and S. Dustdar, "A Privacy Preserving System for AI-assisted Video Analytics," in *2021 IEEE 5th Int. Conf. on Fog and Edge Computing (ICFEC)*, 2021, pp. 74–78. Available: https://ieeexplore.ieee.org/document/9458893.

[7] World Wide Web Consortium (W3C), "WebRTC: Real-Time Communication in Browsers," W3C Recommendation, Jan. 2021. [Online]. Available: https://www.w3.org/TR/webrtc/.

[8] I. F. Akyildiz, A. Kak, and S. Nie, "6G and Beyond: The Future of Wireless Communications Systems," *IEEE Access*, vol. 8, pp. 133995–134030, 2020. (Discusses Edge AI latency benefits).

[9] V. Bazarevsky *et al.*, "BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs," *arXiv preprint arXiv:1907.05047*, 2019. (The seminal paper for the model used).

[10] TensorFlow Team, "TensorFlow.js: Machine Learning for the Web and Node.js," *SysML Conference*, 2018. [Online]. Available: https://js.tensorflow.org/.

[11] A. Papoutsaki *et al.*, "WebGazer: Scalable Webcam Eye Tracking Using User Interactions," in *Proc. of the 25th Int. Joint Conf. on Artificial Intelligence (IJCAI)*, 2016, pp. 3839–3845.

[12] R. Mizrahi *et al.*, "Can Large Language Models Automatically Score Proficiency of Written Essays?," in *Proc. of the 2024 Joint Int. Conf. on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*, 2024, pp. 2758–2771.

[13] M. Fowler and J. Lewis, "Microservices: a definition of this new architectural term," *MartinFowler.com*, 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html. (Industry Standard Definition).

[14] Internet Engineering Task Force (IETF), "JSON Web Token (JWT)," RFC 7519, May 2015. [Online]. Available: https://tools.ietf.org/html/rfc7519.

[15] MongoDB Inc., "MongoDB Architecture Guide," *MongoDB White Paper*, 2024. [Online]. Available: https://www.mongodb.com/collateral/mongodb-architecture-guide.

[16] FastAPI, "FastAPI: High Performance, Easy to Learn, Fast to Code, Ready for Produc-

tion," [Online]. Available: https://fastapi.tiangolo.com/.

[17] P. Author *et al.*, "Desktop Based Edge AI Proctoring System," *Int. J. for Research in Applied Science & Eng. Tech. (IJRASET)*, vol. 14, no. 1, Jan. 2026.

[18] S. Gupta *et al.*, "Automated Proctoring Systems: A Review of Privacy and Security Concerns," *IEEE Access*, vol. 9, pp. 12345–12360, 2021.